

Python String Functions

Practice Assignment · 11 Questions

Solve each question using **string methods only** — slicing, strip(), split(), join(), replace(), the case converters and the is*() validators. Match the expected output **exactly**, including quotes, spacing and capitalization.

Question 1 · The Strict Username Validator

Write a function that takes a username string as input and validates it. To be valid, the username must be at least **5 characters long** after removing any leading/trailing spaces, and it must contain **only letters and numbers** (no special characters like @, _, or spaces inside). If it is valid, return the clean, strictly **lowercase** version of the username. If it is invalid, return the string "Invalid".

INPUT	EXPECTED OUTPUT
Case 1	
" CodeMaster99 "	"codemaster99"
Case 2	
" Python_Dev "	"Invalid"

Question 2 · The Spy Code Reverser

You intercept a secret code where words are separated by hyphens (-). To decode it, you must **reverse the order of the words**, convert every letter to **UPPERCASE**, and replace the hyphens with spaces.

INPUT	EXPECTED OUTPUT
secret_code = "charlie-bravo-alpha"	"ALPHA BRAVO CHARLIE"

Question 3 · The Messy Database Cleaner

You have received a raw string of names from a database exported as a semicolon-separated list. The data is messy, full of random spaces, and has inconsistent casing. Write a function to clean this string so that every name is properly **Title Cased**, extra spaces are removed, and the names are joined into a single string separated by a comma and a space.

INPUT	EXPECTED OUTPUT
raw_names = " john doe ; jAnE sMiTh; ALAN tUrInG "	"John Doe, Jane Smith, Alan Turing"

Question 4 · The Multiple Space Eliminator

A user has accidentally typed a sentence with multiple random spaces between words, and the casing is entirely lowercase. Write code that removes all the extra spaces so there is **exactly one space** between each word, and capitalizes **only the very first letter** of the entire sentence.

Note: Do not use the `re` module — solve it strictly using string functions.

INPUT	EXPECTED OUTPUT
<pre>text = " python is a very powerful language "</pre>	<pre>"Python is a very powerful language"</pre>

Question 5 · The Palindrome Sentence Checker

Determine if a given sentence is a palindrome. To do this correctly using only string methods, remove all spaces and non-alphanumeric characters, convert everything to lowercase, and check whether it reads the same forwards and backwards.

INPUT	EXPECTED OUTPUT
Case 1	
<pre>sentence = "Was it a car or a cat I saw?"</pre>	<pre>True</pre>
Case 2	
<pre>sentence = "Python is fun"</pre>	<pre>False</pre>

Question 6 · The Hashtag Extractor

Given a string representing a social media post, extract all the hashtags. The extracted hashtags must be completely **lowercase**, and the `#` symbol should be removed. Store them in a list. Ignore any words that are not hashtags.

INPUT	EXPECTED OUTPUT
<pre>post = "Loving the SUNSET! #Nature #Sunset_Vibes #beautiful"</pre>	<pre>['nature', 'sunset_vibes', 'beautiful']</pre>

Question 7 · The File Extension Finder

You are given a single comma-separated string containing various filenames. Extract just the file extensions (**without the dot**), ensure there are **no duplicates**, and return them as a list of strings strictly in **UPPERCASE**.

INPUT	EXPECTED OUTPUT
<pre>filenames = "report.pdf, image.JPG, data.csv, notes.txt, photo.jpg"</pre>	<pre>['PDF', 'JPG', 'CSV', 'TXT']</pre>

Question 8 · The Domain Name Slicer

Given an email address string, extract just the company/domain name — the part immediately after the @ symbol and before the dot suffix like .com or .edu. Convert the extracted domain to **Title Case**. If the string does not contain an @, return "Invalid".

INPUT	EXPECTED OUTPUT
Case 1	
email = "developer_99@microsoft.com"	"Microsoft"
Case 2	
email = "admin_at_website.org"	"Invalid"

Question 9 · The Anagram Detective

Write a function to check if two given strings are **anagrams** of each other — they contain the exact same letters in the exact same frequencies, just rearranged. Ignore any spaces and treat uppercase and lowercase letters as the same.

Note: Solve it using string methods and basic built-in functions (like sorted()).

INPUT	EXPECTED OUTPUT
str1 = "Dormitory" str2 = "Dirty room"	True

Question 10 · The Messy Phone Number Formatter

You are building a CRM system and user-submitted phone numbers are a mess — they contain spaces, parentheses, periods, and hyphens. Write a function that takes a raw phone number string and cleans it by removing all non-digit characters. Once cleaned, validate the number:

- If the cleaned string contains **exactly 10 digits**, format and return it as "(XXX) XXX-XXXX".
- If it contains anything other than 10 digits, return "Invalid Number".

*Note: Case 1 fails because after removing symbols it has **11** digits, due to the leading "1".*

INPUT	EXPECTED OUTPUT
Case 1	
raw_num = " +1 (555) 123.4567 "	"Invalid Number"
Case 2	
raw_num = "555-987-6543"	"(555) 987-6543"

Question 11 · The Product SKU Inspector

An inventory system requires product codes (SKUs) to strictly match this format after being cleaned up: exactly **3 uppercase letters**, a single hyphen, and exactly **4 digits** (e.g. "ABC-1234").

Write a function that receives a messy SKU string. First clean it by stripping leading/trailing spaces and converting it to uppercase. Then validate it using string methods. Return "Valid: [Cleaned_SKU]" if it passes, or "Invalid SKU" if it fails any formatting rule.

Note: Use `isalpha()`, `isdigit()` etc. — no `re` module. Case 2 fails (2 letters, 5 digits); Case 3 fails (special character).

INPUT	EXPECTED OUTPUT
Case 1	
sku = " xYz-9876 "	"Valid: XYZ-9876"
Case 2	
sku = "AB-12345"	"Invalid SKU"
Case 3	
sku = "A@C-1234"	"Invalid SKU"